

AI-Based Road User Detection Using Traffic Monitoring Radar

Ármin Ihász
Gábor Tompos

May 25, 2026

1 Literature Review

1.1 Radar Point Cloud

1.1.1 1.1 What quantities does one radar detection point carry? What is the physical meaning of each?

A radar point usually contains range, azimuth angle, Doppler velocity, signal intensity, and sometimes elevation. Range describes the distance between the radar and the reflection point. The azimuth angle gives the horizontal direction of the target relative to the radar. Doppler velocity measures the relative radial speed of the object, which helps separating moving and static objects. Signal intensity represents the reflected energy and often depends on object material, angle, and distance. Some radars also estimate elevation, but in our setup the main focus was the BEV plane. Together these quantities form a sparse point cloud representation of the environment.

1.1.2 1.2 What does “sparsity” mean in the context of radar point clouds, and why is it a fundamental challenge for machine learning?

Radar point clouds are sparse because objects are represented only by a few reflection points instead of dense textures like in camera images. A pedestrian may produce only a few detections in one frame, while even cars often appear fragmented. This makes object boundaries uncertain and noisy. Sparse inputs are difficult for CNN-based models because small spatial changes can strongly affect the learned features. Sparsity also increases sensitivity to noise, ghost detections, and missed reflections. In our project this was especially visible for pedestrians and cyclists, therefore temporal accumulation and frame merging were important preprocessing steps.

1.2 Camera-Radar Synchronization for Annotation

1.2.1 2.1 Cameras and radars do not record the same physical features. Where do reflection points appear on a car in BEV?

The radar reflections usually appeared around strong reflective surfaces of the vehicle, mainly near corners, wheels, or metallic edges. Because the radar was mounted overhead at around 3.5 m height, the reflected points did not always align perfectly with the visual center of the car seen by the camera. Our ground-truth boxes were drawn around the full vehicle footprint in BEV space, while the radar points often covered only smaller regions. This mismatch introduced annotation uncertainty, especially for partially occluded or distant vehicles.

1.2.2 2.2 What is a homography and why do we need it in this project?

Homography is a geometric transformation that maps points between two planes using a projective transformation matrix. In this project it was used to transform camera detections into the radar BEV coordinate system. Without homography the camera bounding boxes and radar point clouds would not share the same spatial reference frame. We tested both RANSAC-based estimation and Direct Linear Transformation (DLT). Finally DLT was chosen because it produced more stable mappings for our fixed overhead setup.

1.2.3 2.3 Main sources of error when transferring camera labels to radar BEV images

The main error sources were synchronization mismatch, imperfect homography estimation, radar sparsity, and physical differences between optical and radar sensing. Radar reflections often did not match the visual contour of objects. Small calibration errors caused shifted annotations in BEV space. Occlusions and low reflection targets such as pedestrians also increased uncertainty. Additionally, distant objects produced more scattered reflections because of angular resolution limitations.

1.2.4 2.4 Practical mitigation strategies

We reduced synchronization issues using synced timestamps between camera and radar frames. DLT homography calibration was refined manually using reference points in the scene. Temporal accumulation and frame merging helped stabilize sparse radar reflections. Ghost detections were filtered using confidence thresholds and simple postprocessing. We also merged pedestrian+bike cases into the cyclist class to reduce ambiguous labels and improve consistency.

1.3 Grid Mapping

1.3.1 3.1 What information can be encoded into the channels of a BEV pseudo-image?

A BEV pseudo-image may contain radar intensity, Doppler velocity, point density, occupancy, or temporal accumulation information. Different channels can represent different physical quantities similarly to RGB image channels. In our implementation the grid mainly encoded accumulated radar reflections after preprocessing and filtering.

1.3.2 3.2 Additional pseudo-image channels

One useful additional channel could be temporal consistency, which stores how often a cell remained occupied across consecutive frames. This may help suppress random noise. Another possible channel is Doppler variance, which describes motion stability inside a cell. This could help distinguishing moving pedestrians from static background clutter.

1.3.3 3.3 Trade-off when choosing grid cell resolution

Smaller cells provide finer spatial detail but create sparser images and higher computational cost. Larger cells reduce sparsity and noise, but object shapes become less precise. We selected a compromise resolution where vehicles remained visible while pedestrians still produced detectable structures.

1.3.4 3.4 Why accumulate several frames?

Accumulating multiple frames increases the number of radar points and reduces sparsity. This is especially important for pedestrians, which often generate very few reflections in a single frame. However, accumulating too many frames may blur moving objects and introduce motion artifacts. In our final pipeline five-frame accumulation gave the best compromise.

1.4 Object Detection with YOLO on Radar BEV Images

1.4.1 4.1 Why does YOLO provide latency advantages?

YOLO performs object detection in a single forward pass, making it significantly faster than two-stage detectors such as Faster R-CNN. This low latency is important for real-time traffic monitoring and live inference demonstrations. The simpler architecture also reduced computational requirements during testing.

1.4.2 4.2 Why is YOLO practical for BEV pseudo-images?

Although BEV maps are not natural photographs, they still form structured image-like tensors that CNNs can process efficiently. YOLO is practical because it handles spatial patterns well and supports fast inference. Fine-tuning pretrained models was preferable because the dataset size was limited and training from scratch would likely overfit.

1.4.3 4.3 Downside of adding extra BEV channels

Adding more channels increases model complexity and may reduce compatibility with pretrained weights. With limited data this can increase overfitting risk. Therefore extra channels should only be added if they clearly improve validation performance or robustness.

1.4.4 4.4 What preprocessing steps improved convergence in the paper?

The paper used grid mapping to transform sparse radar points into dense image-like BEV representations. Temporal accumulation and feature encoding also helped improve convergence. These preprocessing steps reduced sparsity and made the radar input more suitable for CNN-based object detection.

1.4.5 4.5 Differences between our setup and the paper

Our setup used a fixed overhead 24 GHz radar, while the paper used a vehicle-mounted 77 GHz automotive radar. The overhead setup changed reflection geometry and reduced ego-motion effects. The lower frequency radar also produced different spatial resolution characteristics. Finally, our system relied heavily on stationary traffic monitoring and temporal accumulation, unlike the moving-vehicle scenario in the paper.

1.4.6 4.6 Two preprocessing ideas for our use-case

One preprocessing idea is adaptive temporal accumulation, where distant objects are accumulated longer because they generate fewer reflections. Another idea is confidence-based ghost filtering, which removes unstable detections before grid generation. Both methods target sparsity and annotation mismatch problems.

1.4.7 4.7 False positive or false negative?

In traffic monitoring, false negatives are usually more critical because missing an object may lead to unsafe decisions or incorrect statistics. However, too many false positives also reduce system reliability. The balance can be adjusted using confidence thresholds, IoU thresholds, and postprocessing filters.

1.5 Implications for Our Pipeline

1.5.1 5.1 Checklist for the pipeline

The pipeline should verify synchronization quality, homography accuracy, radar filtering, frame accumulation settings, grid resolution, annotation consistency, and dataset balance. During training it is important to monitor overfitting, validation metrics, and inference speed. Postprocessing should also check for ghost detections and unstable predictions.

2 Introduction

The goal of this project was building an AI-based traffic participant detection pipeline using synchronized radar and camera measurements. The main objective was detecting pedestrians, cars and cyclists directly from radar-generated Bird's-Eye View pseudo-images.

Compared to a normal computer vision task, this problem turned out to be much harder than we originally expected. In a normal RGB image, objects have visible shapes, edges and textures. In radar data however, objects only appear as sparse reflection points. Cars usually generate larger and more stable clusters, while pedestrians often only appear as a few weak points. Cyclists are also difficult because the bicycle and rider may appear as partially separated structures.

Because of this, the biggest challenge of the project was not the neural network itself, but the preprocessing and dataset generation pipeline. A large part of the work involved experimenting with:

- homography mapping,
- radar-camera synchronization,
- temporal accumulation,
- radar BEV generation,
- cyclist merge logic,
- oversampling,
- and ghost detection filtering.

A lot of engineering decisions had to be tested experimentally. In many cases, the theoretically more advanced method was not necessarily the best solution for the actual measurement setup.

3 Measurement Setup and Dataset

The measurements were recorded using a fixed overhead traffic monitoring radar together with a synchronized camera. The camera was mainly used for generating labels and validating detections, while the final detector itself only worked on radar-generated BEV pseudo-images.

The useful monitored region covered approximately:

$20\text{ m} \times 70\text{ m}$

The radar data contained:

- reflection point coordinates,
- amplitude information,
- Doppler values,
- timestamps,
- and frame identifiers.

Several different recording days were available. However, the onsite measurements from 20260506 became especially important later because they were the closest to the final expected benchmark and demo environment.

The final object classes were:

- pedestrian,
- car,
- cyclist.

The dataset was heavily imbalanced. Cars dominated the dataset, pedestrians were significantly fewer and cyclists were relatively rare.

4 General Pipeline

The final system was not simply a YOLO training script, but a full radar AI processing pipeline.

Conceptually, the workflow looked as follows:

Camera video $\rightarrow$ YOLO annotation $\rightarrow$ Tracking $\rightarrow$ Homography mapping

$\rightarrow$ Radar synchronization $\rightarrow$ Radar frame accumulation $\rightarrow$ BEV generation

$\rightarrow$ YOLO dataset creation $\rightarrow$ Training $\rightarrow$ Realtime inference.

Most of the actual complexity was inside the preprocessing stages rather than the neural network training itself.

5 Camera Annotation and YOLO Model Selection

The first stage was generating camera-side annotations. Several YOLO versions were tested before selecting the final setup.

The comparison is shown in Table 1.

Table 1: Comparison of tested YOLO versions.

YOLO version	Param. count	Layers	Total det.	Net det.	Speed (it/s)
YOLOv8	25,902,640	169	37,147	12,893	19.96
YOLOv11	20,114,688	231	33,934	13,413	20.17
YOLOv12	20,201,216	292	41,132	13,634	16.42

YOLO11 was finally selected because it gave the best practical balance between:

- inference speed,
- parameter count,
- stability,
- and overall useful detections.

YOLO12 produced slightly more detections, but inference became slower, while YOLO8 had a significantly larger parameter count.

For camera annotation a larger YOLO11 model was used because annotation quality was important. For the final radar detector however, YOLO11n was preferred because:

- the radar dataset was relatively small,
- larger models overfit more easily,
- realtime inference was important.

The annotation stage intentionally used relatively permissive confidence thresholds. Missing an object during annotation would permanently remove it from the training dataset, while false positives could still be filtered later.

6 Homography Mapping

After generating camera detections, the labels had to be projected into radar coordinates. This was done using homography mapping.

The bottom-center point of each bounding box was projected because this approximates the physical contact point with the road plane.

Two approaches were tested:

- Direct Linear Transformation (DLT),
- RANSAC-based homography.

Initially RANSAC seemed promising because it can reject incorrect calibration points automatically. However, in practice the manually selected calibration points were already relatively accurate and the camera setup was fixed.

Because of this, RANSAC sometimes distorted the mapping unnecessarily. The measured statistics were:

$$\text{inliers} = 8/9$$

with mean reprojection error:

$$1.6711\text{ m}.$$

Finally DLT was selected because it produced more stable projections for the actual setup.

7 Cyclist Merge and Annotation Postprocessing

Cyclists caused a surprisingly difficult problem during annotation.

The camera detector often produced separate pedestrian and bicycle detections for the same cyclist. While this behaviour makes sense for standard image detection, it was not ideal for radar training because in radar data the rider and bicycle usually appear as one combined reflection cluster.

To solve this, a custom cyclist merge step was implemented.

The algorithm checked:

- overlap between bicycle and pedestrian boxes,
- spatial proximity,
- and padded overlap regions.

If the conditions were satisfied, the detections were merged into one cyclist label.

This worked much better than simply using nearest-neighbour matching because overlap provided a more stable indicator in perspective-distorted camera images.

Additional postprocessing steps were also necessary:

- parked vehicle filtering,
- road-mask filtering,
- ghost detection filtering,
- radar-supported validation.

Without these filtering steps the dataset contained many unstable or physically unrealistic labels.

8 Radar-Camera Synchronization

The radar and camera streams were not perfectly synchronized, therefore timestamp-based matching was required.

Nearest-neighbour timestamp synchronization was used between radar frames and camera detections.

This step turned out to be very important because even small temporal shifts could move labels away from the actual radar reflection clusters. This was especially problematic for pedestrians and cyclists.

After synchronization, labels were additionally validated using nearby radar support. If a projected label had no nearby radar reflections, it was removed from the final dataset.

This significantly improved the physical consistency of the training labels.

9 Ghost Detection Filtering

Ghost detections became one of the largest practical problems during preprocessing.

These included:

- parked vehicles,
- static clutter,
- reflections,
- unstable camera detections,
- isolated radar noise points.

Several filtering stages were implemented.

Known parked vehicle regions were masked, labels outside the useful road region were removed and labels without radar-supported reflections were discarded.

Because of this, preprocessing and postprocessing became one of the most important engineering parts of the entire project.

10 Radar BEV Generation

The raw radar point cloud was converted into Bird’s-Eye View pseudo-images. This became the final representation used for training.

The BEV image contained three channels:

- occupancy density,
- normalized log-amplitude,
- Doppler velocity.

The occupancy channel represented the density of radar points inside each grid cell.

The amplitude channel required normalization because radar reflections strongly depend on distance. Radar power approximately follows:

$$|E_{hh}|^2 \propto \frac{1}{R^4}$$

therefore distant objects appear much weaker than nearby ones. To compensate for this, range normalization and logarithmic scaling were applied. The Doppler channel encoded radial velocity:

$$v = (FG - 32) \cdot 1.4359$$

followed by nonlinear scaling to make weak pedestrian motion more visible.

11 Temporal Accumulation

Temporal accumulation became one of the most important ideas of the whole project.

A single radar frame often contained too few reflection points, especially for pedestrians. In many cases pedestrians only generated 2-3 visible points in one frame, which is not enough for stable object learning.

Because of this, multiple radar frames were merged before generating one BEV pseudo-image.

In the completed Stage 1 setup:

- 5 consecutive radar frames were accumulated,
- every 5th accumulated frame became one dataset sample.

This significantly improved visibility of weak radar targets while keeping dataset size manageable.

Later, newer experiments also tested:

- 10-frame accumulation,
- overlapping export windows,
- exporting every 2nd frame.

This created denser temporal information and more overlapping radar samples.

12 Oversampling Strategy

The onsite recordings from 20260506 were intentionally oversampled during training because these recordings were closest to the expected final demo environment.

The oversampling was not simply random duplication. Instead, overlapping temporally accumulated radar samples were generated from nearby frame windows.

This allowed the model to observe slightly different temporal structures even around very similar physical scenes.

At the same time, care had to be taken to avoid excessive validation leakage because overlapping accumulated samples can become extremely similar.

The oversampling mainly helped improve difficult pedestrian and cyclist cases.

13 Training Strategy

Originally the training pipeline was designed as a two-stage system.

The idea behind this was separating:

- general radar representation learning,
- and domain-specific adaptation.

Stage 1 focused on learning the overall structure of radar BEV pseudo-images and general radar object behaviour using all available recordings.

The goal was not maximizing benchmark performance on one specific day, but rather learning:

- radar occupancy patterns,
- Doppler behaviour,
- typical object structures,
- and general radar representations.

Stage 2 was planned as a domain adaptation step.

The idea was further fine-tuning mainly on the onsite recordings closest to the final benchmark and demo conditions.

This separation was useful because different measurement days contained:

- different clutter,
- different traffic density,
- different parked vehicles,
- and slightly different radar behaviour.

The final completed results shown in this report mainly correspond to the Stage 1 setup. Additional experiments with modified accumulation and validation setup were still running at the time of writing.

Several standard RGB augmentations were also disabled because radar pseudo-images are physically very different from normal images.

For example:

- vertical flip would invert Doppler meaning,
- strong mosaic augmentation created unrealistic radar structures,
- large translations could break spatial consistency.

14 Training Results

The Stage 1 training curves are shown in Figure 1.

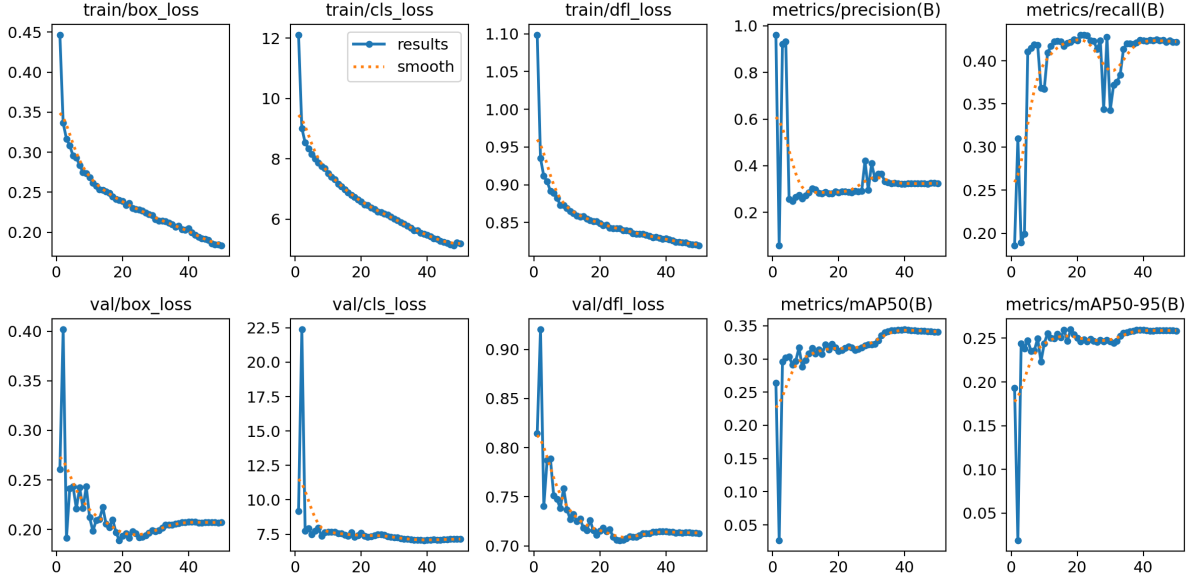


Figure 1: Training and validation curves of the Stage 1 model.

The losses decreased relatively consistently, showing that the model learned meaningful radar structures from the generated BEV pseudo-images.

The final performance reached approximately:

$$mAP50 \approx 0.34$$

and:

$$mAP50-95 \approx 0.26.$$

Considering the sparsity and difficulty of radar-only detection, these results were still relatively encouraging.

15 Precision, Recall and Confusion Analysis

The precision-recall curves are shown in Figure 2.

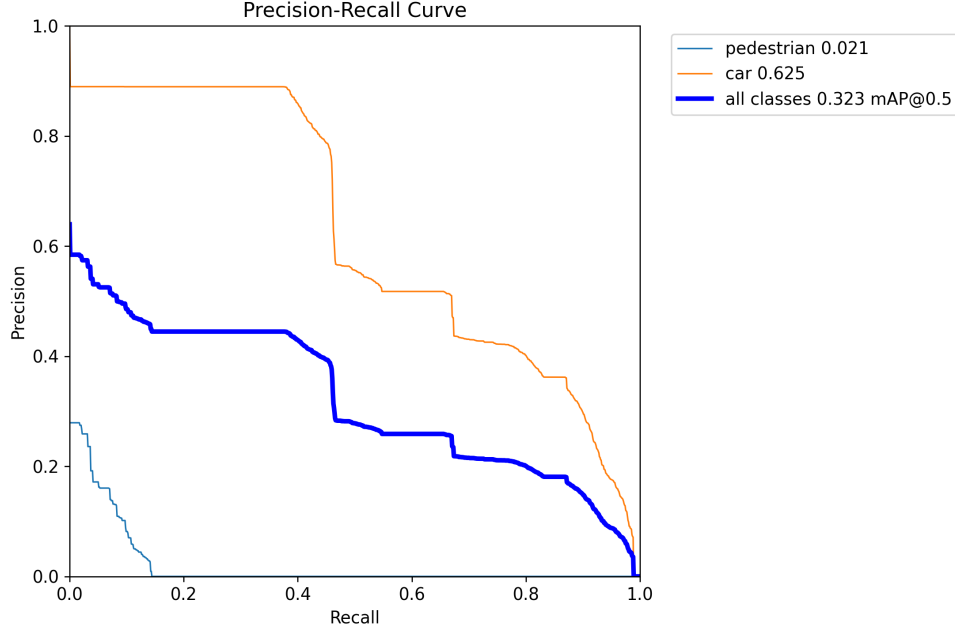


Figure 2: Precision-recall curve of the Stage 1 detector.

Cars were detected much more reliably than pedestrians. This is expected physically because cars produce larger and denser radar reflection clusters.

Pedestrians remained difficult because they often only appeared as a few weak radar points.

The best F1 score was approximately:

$$F1 \approx 0.31$$

at relatively low confidence threshold.

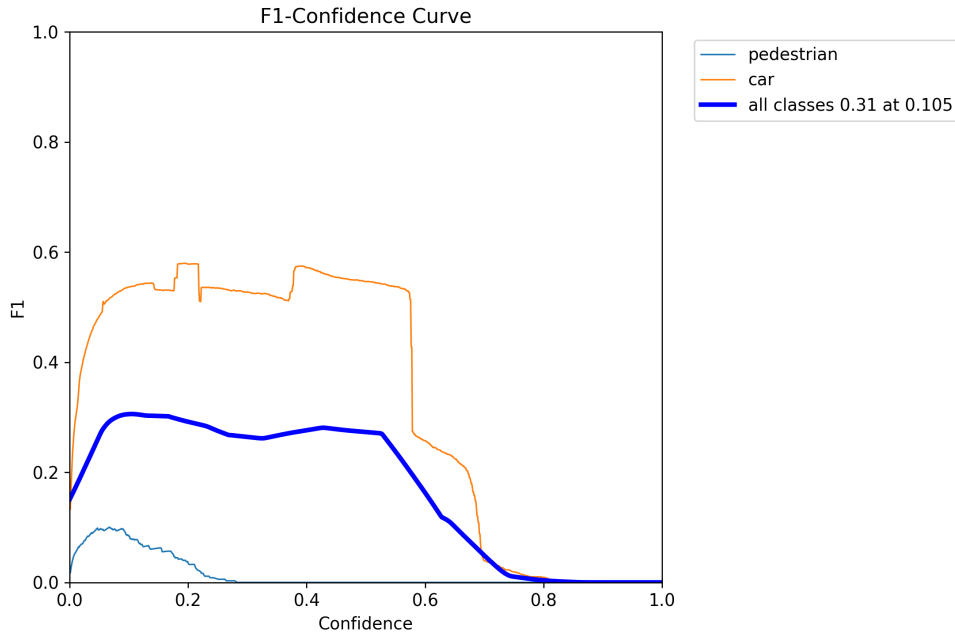


Figure 3: F1-confidence curve.

The confusion matrices also confirmed this behaviour.

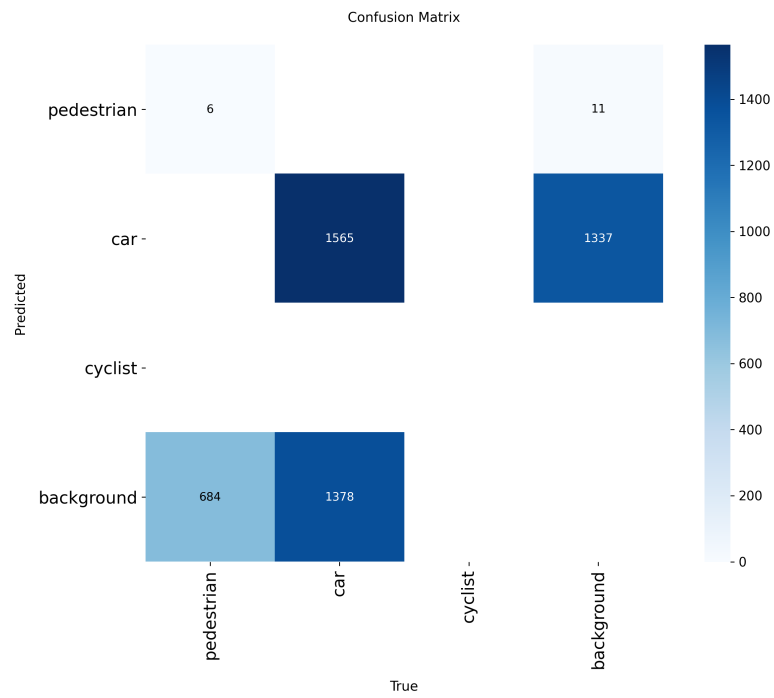


Figure 4: Raw confusion matrix.

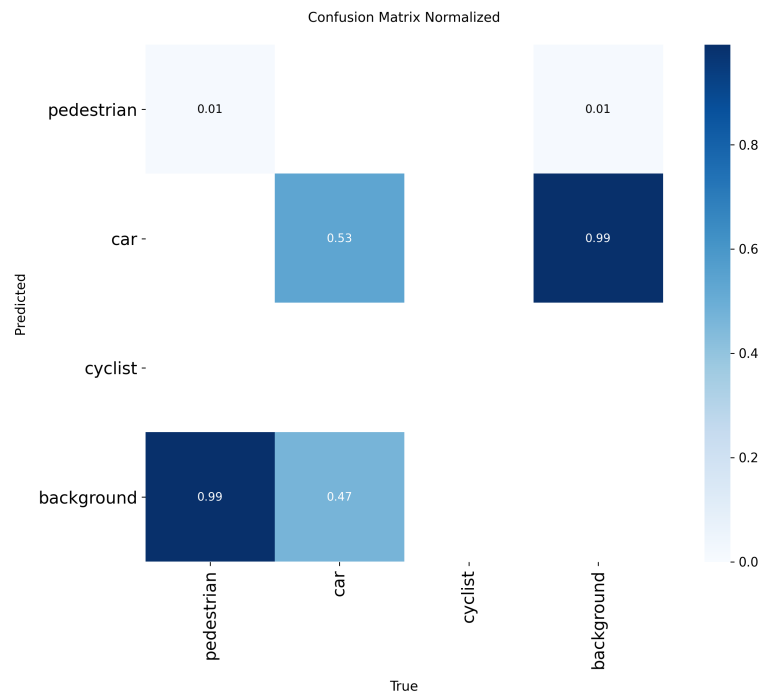


Figure 5: Normalized confusion matrix.

The main issue remained pedestrian-background confusion because pedestrian radar returns are weak and sparse.

16 Generated Dataset Examples

The following figures show examples of the generated radar BEV pseudo-images and YOLO labels.

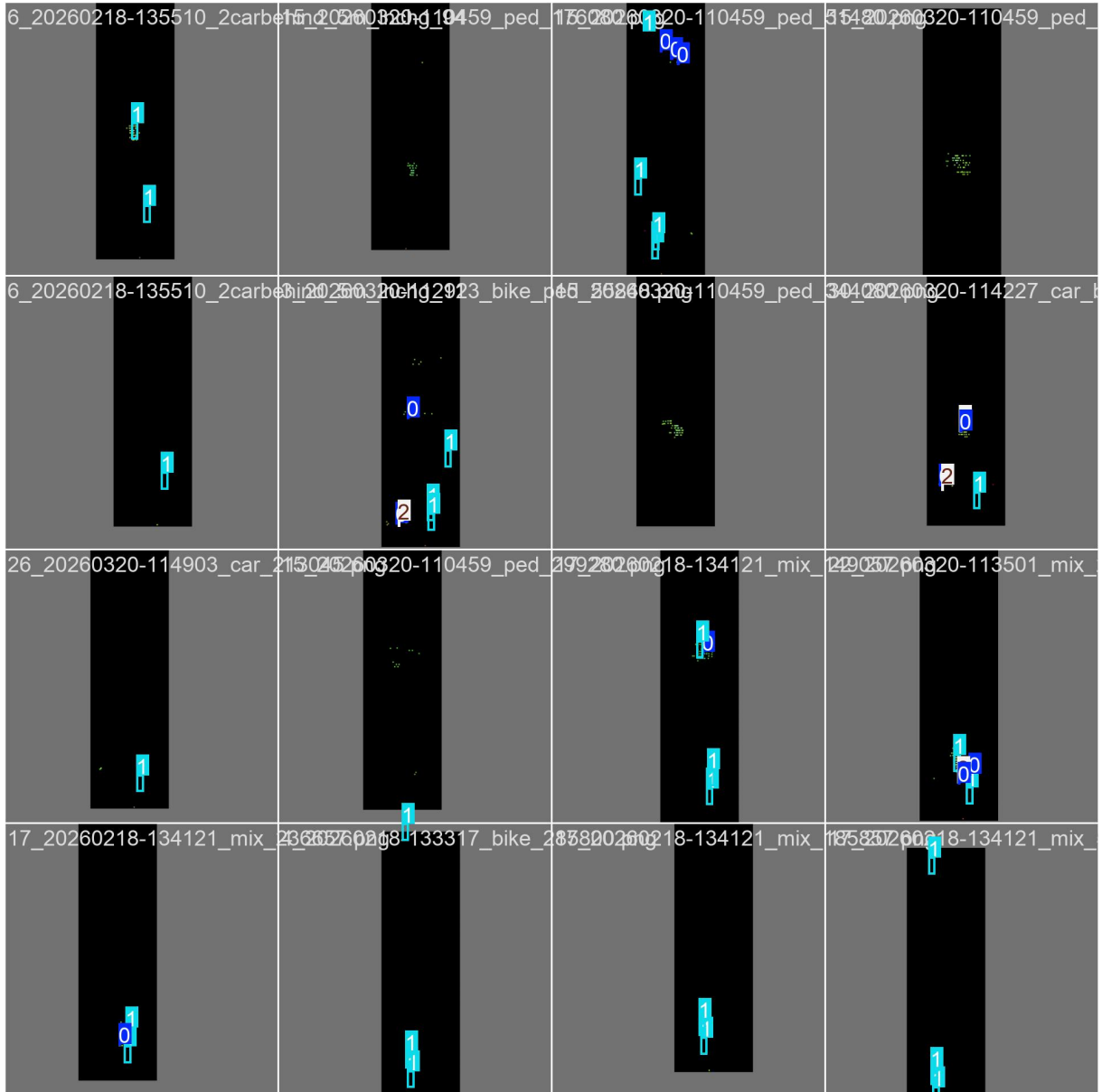


Figure 6: Generated radar BEV training samples.

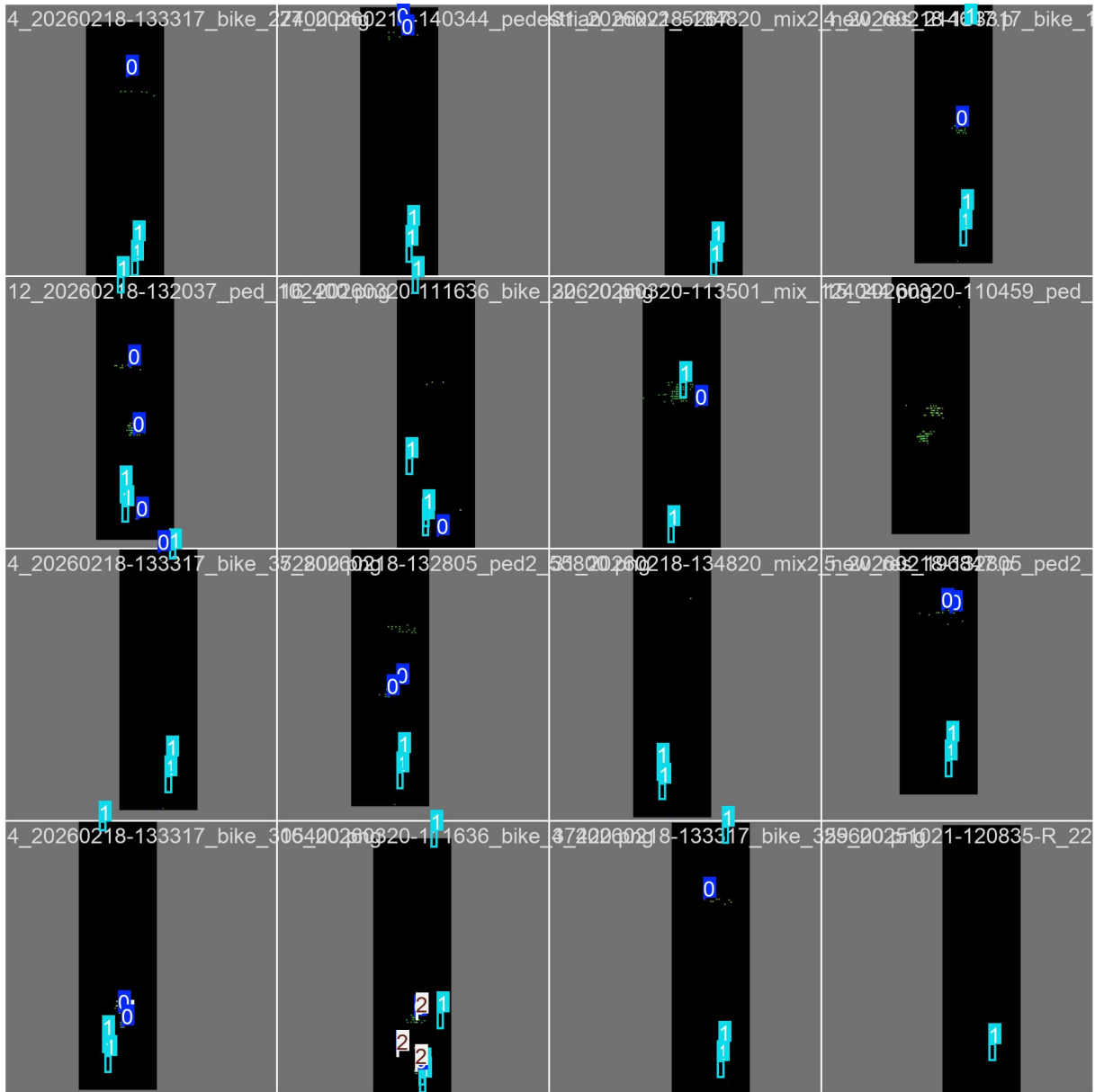


Figure 7: Additional generated training samples.

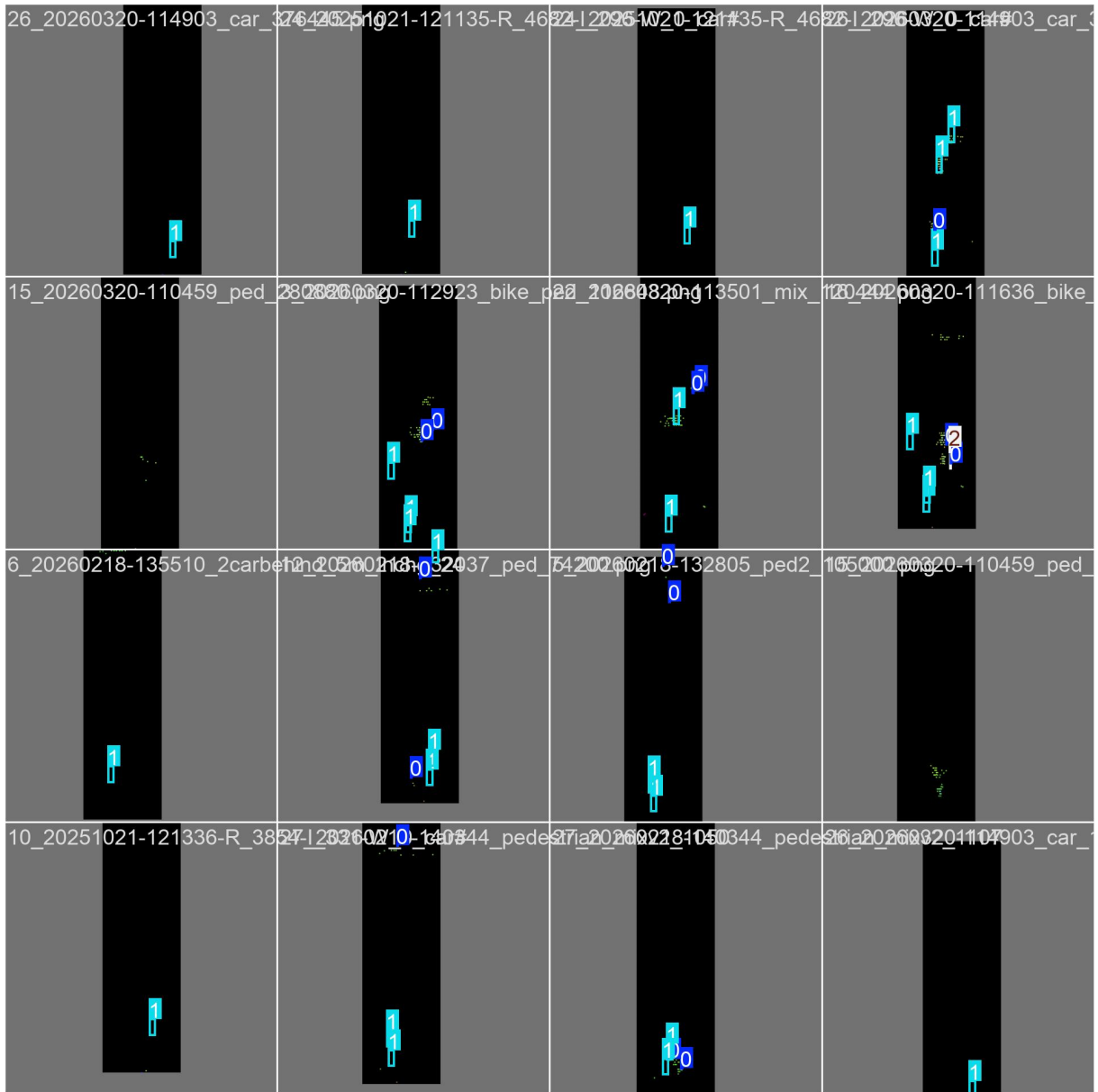


Figure 8: Sparse radar detections and generated labels.

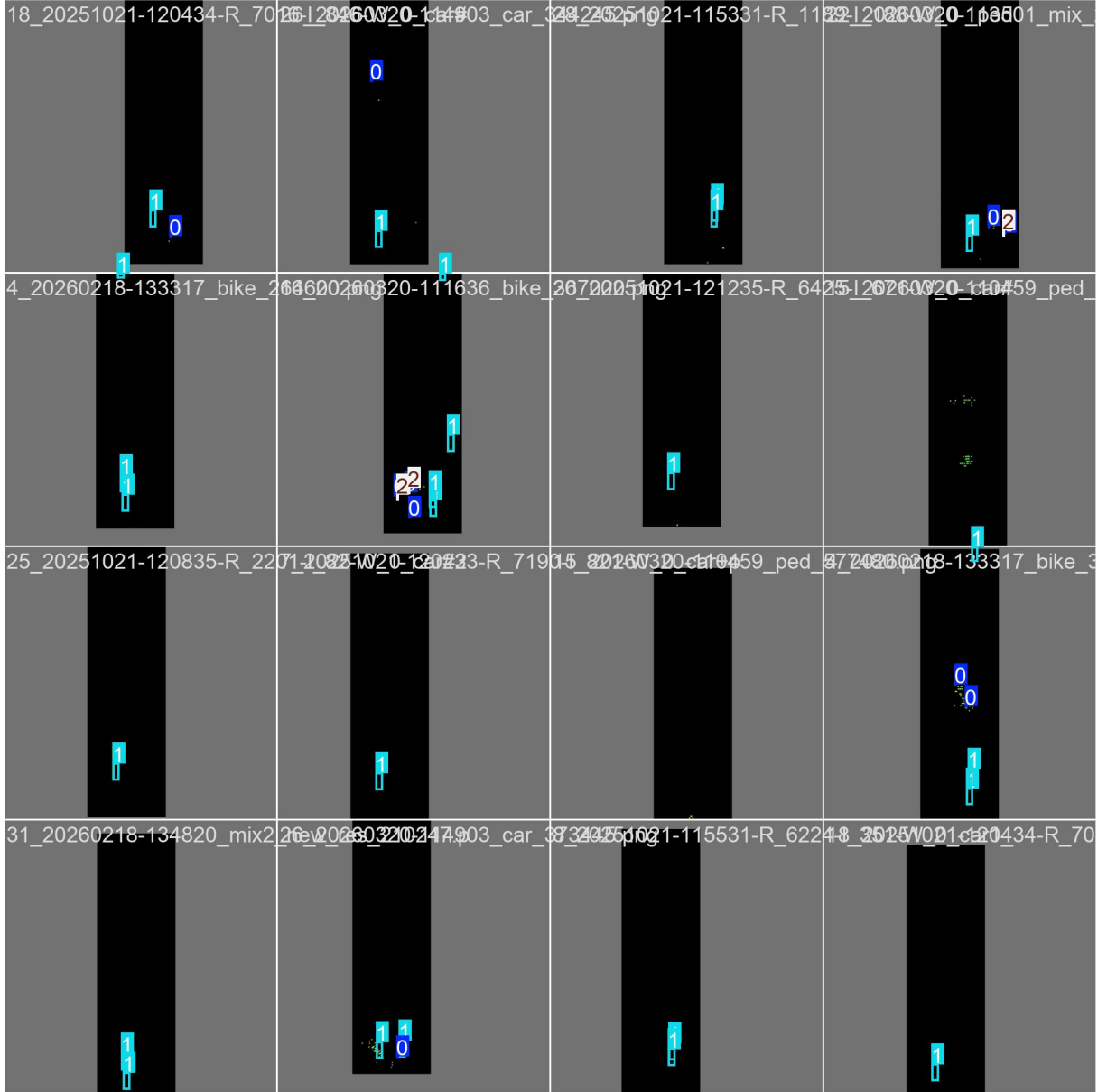


Figure 9: Later generated BEV training batch.

These examples clearly show why radar detection is difficult. Cars usually generate strong stable clusters, while pedestrians often only appear as a few scattered reflections.

17 Inference Rehearsal

After training, the best model was exported into ONNX format for inference testing and deployment rehearsal.

An inference rehearsal sequence was also created where:

- one side displayed the radar inference output,
- while the other side showed synchronized camera annotations.

This allowed direct visual comparison between:

- radar detections,
- generated labels,
- and camera-side ground truth behaviour.

The rehearsal also verified that the entire end-to-end pipeline worked correctly:

Radar $\rightarrow$ BEV generation $\rightarrow$ YOLO inference $\rightarrow$ Visualization.

18 Discussion

Looking back at the whole project, probably the most important thing we learned is that radar AI problems are very different from standard computer vision tasks. At the beginning we expected that the neural network itself would be the hardest part, but in reality most of the work and experimentation went into the preprocessing pipeline and dataset generation.

One of the biggest challenges was the sparsity of the radar data. Cars usually produced relatively stable and dense reflection clusters, but pedestrians often only appeared as a few weak points in a single frame. Because of this, many of the decisions during the project focused on improving temporal consistency and visibility of these weaker targets.

Temporal accumulation became one of the most important parts of the entire system. Merging multiple radar frames together made pedestrians significantly easier to see in the BEV representation. At the same time, too much accumulation could smear moving objects and reduce temporal precision, therefore finding a balance required a lot of testing.

Another interesting part was that many standard computer vision ideas did not work particularly well here. Some normal RGB augmentations produced completely unrealistic radar patterns, so augmentations had to be selected much more carefully. The same was true for cyclist handling and annotation generation. In normal RGB images, separating bicycles and pedestrians makes sense, but in radar they often appear as one combined structure, therefore cyclist merging produced more physically meaningful labels.

The homography mapping was also more difficult than originally expected. RANSAC was tested because theoretically it should improve robustness, but in practice the fixed camera setup and manually selected calibration points made the simpler DLT solution more stable overall. This became a good example where the theoretically more advanced solution was not necessarily the best engineering choice for the actual environment.

Oversampling the onsite recordings from 20260506 also turned out to be useful. Since these measurements were closest to the expected demo conditions, increasing their representation inside the training set helped the model adapt better to the final environment.

The two-stage training strategy was also important conceptually. Stage 1 focused more on general radar representation learning, while Stage 2 was intended for domain adaptation and benchmark-specific fine-tuning. Even though the final completed results mainly came from the Stage 1 setup, this separation still helped structure the experiments and training process better.

Overall, the project demonstrated that radar-only object detection is possible, but preprocessing and physically meaningful dataset generation are at least as important as the neural network architecture itself.

19 Conclusion

In this project, a complete radar-based traffic participant detection pipeline was implemented using synchronized radar and camera measurements.

The system included:

- automatic camera annotation,
- homography mapping,
- radar synchronization,
- temporal accumulation,
- BEV pseudo-image generation,
- YOLO dataset creation,
- and realtime-capable object detection.

The completed Stage 1 model achieved approximately:

$$mAP50 \approx 0.34$$

which demonstrates that meaningful object detection is possible even from sparse radar-only representations.

The project also highlighted that in radar AI systems, preprocessing and physically meaningful dataset generation are extremely important. Simply increasing model size would not solve the problem if the radar representation and labels are unstable or physically inconsistent.